

0	1	2	3	4	5	6
0	1	1	2	3	5	8

ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos de Programação Dinâmica

Aula 5 · resolver uma vez, guardar para sempre

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Qual é a complexidade da pesquisa exaustiva no caixeiro-viajante?

2

Quais são os três passos da divisão e conquista?

3

Qual é a complexidade do Merge Sort?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Tem moedas de 1, 3 e 4 centavos. Qual é a forma de dar 6 centavos de troco com o menor número de moedas?

Estratégia "gulosa": a maior moeda primeiro



A melhor solução



Escolher o que parece melhor agora nem sempre dá o melhor resultado. Hoje vamos aprender a encontrar sempre o ótimo — sem experimentar tudo.

Não resolver duas vezes o mesmo problema

1

A ideia

Subproblemas que se repetem: o caso de Fibonacci.

2

Duas formas

Bottom-up e top-down (memorização).

3

O princípio

O princípio da optimalidade de Bellman.

4

O troco

Um problema de otimização resolvido passo a passo.

Pergunta-guia: e se, em vez de recalcular, tomássemos nota das respostas?

1

PARTE 1

A ideia

Subproblemas que se repetem.

Resolver cada subproblema uma só vez

Técnica para problemas com **subproblemas que se sobrepõem**: resolve cada um **apenas uma vez** e **registra o resultado numa tabela**.

Divisão e conquista (Aula 4)

Subproblemas **independentes**.
Cada um é resolvido uma vez, naturalmente.

Ex.: as duas metades do Merge Sort.

Programação dinâmica (hoje)

Subproblemas **sobrepostos**: os mesmos aparecem muitas vezes.
Guardar as respostas evita repetir trabalho.

Ex.: os números de Fibonacci.

"Programação" quer dizer planeamento

anos 50

Richard Bellman

matemático norte-americano

Objetivo: otimizar processos de decisão em várias fases.

1

"Programação" = planeamento

Como em "programar as férias" ou "o programa das festas". Nada a ver com escrever código.

2

De ferramenta matemática a técnica geral

Nasceu na matemática aplicada; na informática tornou-se uma estratégia geral de conceção de algoritmos.

3

Não só para otimização

Aplica-se a muitos problemas, não apenas a tipos especiais de problemas de otimização.

Os números de Fibonacci

F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	F(6)	F(7)	F(8)	F(9)	F(10)
0	1	1	2	3	5	8	13	21	34	55



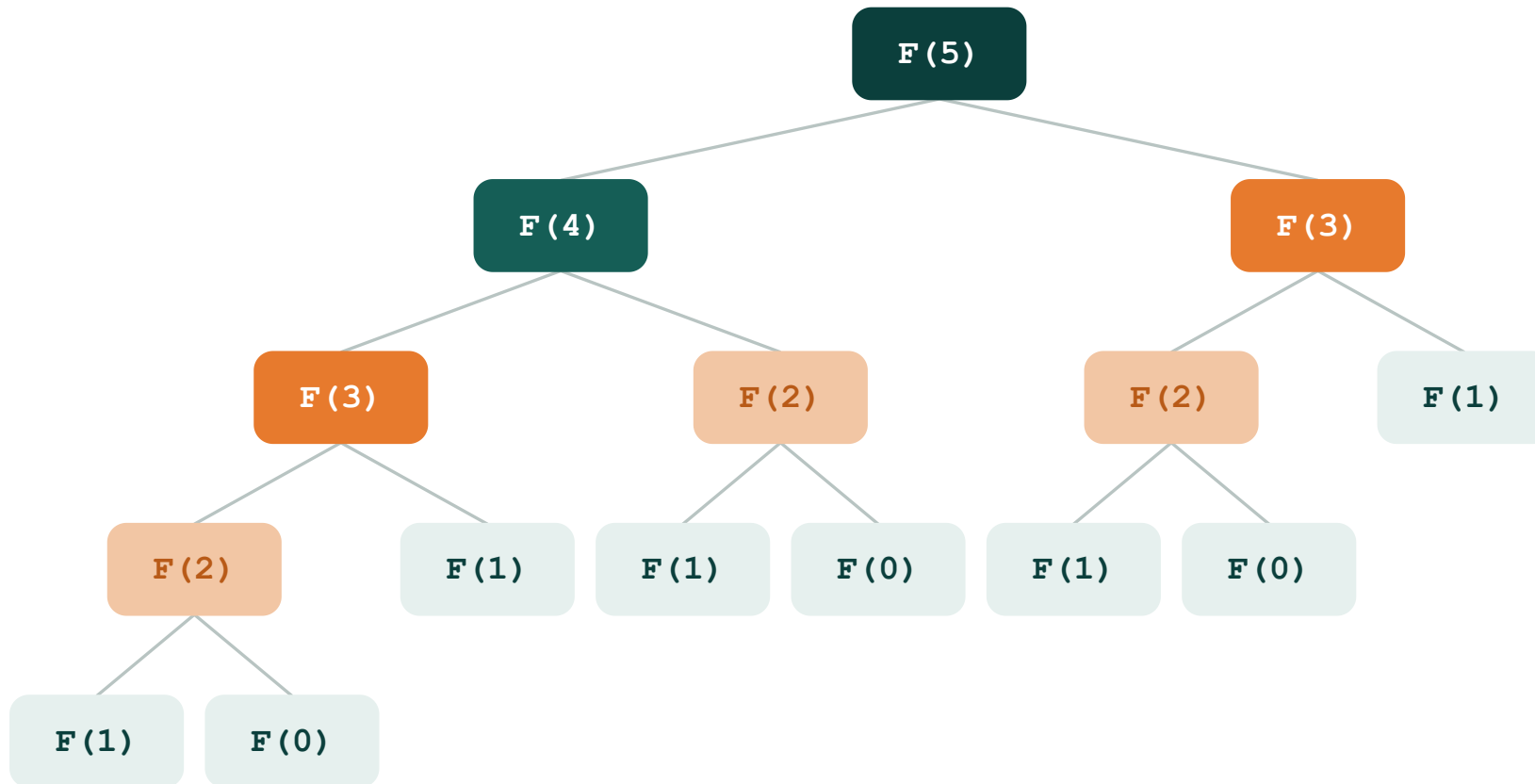
$2 + 3 = 5$

$$F(n) = F(n - 1) + F(n - 2) \text{ para } n > 1$$

$$F(0) = 0, \quad F(1) = 1$$

Casos base a verde escuro; cada caso geral depende de dois subproblemas mais pequenos.

A recursão direta repete trabalho



Para calcular $F(5)$

15

chamadas

$F(3)$ calculado 2 vezes

$F(2)$ calculado 3 vezes

$F(1)$ calculado 5 vezes

Só 6 valores diferentes!

Recursão direta vs tabela

n	Chamadas (recursão direta)	Tempo	Somas (tabela)
10	177	< 1 μ s	9
20	21 891	< 0,1 ms	19
40	331 milhões	0,3 s	39
50	40,7 mil milhões	41 s	49
60	$\approx 5 \times 10^{12}$	$\approx 1,4$ horas	59

Tempos indicativos a 10^9 chamadas por segundo, como na Aula 2.

De exponencial...

...a linear

só por guardar resultados que já tínhamos calculado.

2

PARTE 2

Duas formas de fazer

De baixo para cima, ou de cima para baixo com memória.

Preencher a tabela do mais pequeno para o maior

F[0]	F[1]	F[2]	F[3]	F[4]	F[5]	F[6]	F[7]
0	1	1	2	3	5	8	?

da esquerda para a direita

$$F[7] = F[6] + F[5] = 8 + 5 = 13$$

fib(n) :

```
F[0] ← 0;  F[1] ← 1
para i ← 2 até n:
    F[i] ← F[i-1] + F[i-2]
devolver F[n]
```

Custo

Tempo

$O(n)$

Espaço

$O(n)$

uma tabela com $n + 1$ posições

Recursão... mas com um caderno de notas

```
fibMem(n) :  
  se memo[n] já existe:  
    devolver memo[n]  
  se n < 2: resultado ← n  
  senão: resultado ← fibMem(n-1) + fibMem(n-2)  
  memo[n] ← resultado  
  devolver resultado
```

Consultar antes de calcular; **anotar** depois de calcular.

	Bottom-up	Top-down
Direção	pequeno → grande	grande → pequeno
Técnica	ciclo + tabela	recursão + memória
Subproblemas	resolve todos	só os necessários

Analogia

Um aluno que, antes de resolver um exercício, vai ver ao caderno se já o resolveu.

Gastar memória para poupar tempo

A troca

Tempo: **exponencial** $\rightarrow O(n)$

Espaço: **$O(1) \rightarrow O(n)$**

Aperfeiçoamento

Para calcular o próximo número só precisamos dos dois últimos. Basta guardar duas variáveis: espaço $O(1)$.

Uma "janela" de duas posições que desliza pela sequência:

0	1	1	2	3	5	8	13
0	1	1	2	3	5	8	13

$a + b = \text{próximo}$

depois: $a \leftarrow b; \quad b \leftarrow \text{próximo}$

Espaço: $O(1)$

Para calcular $F(50)$...

Quantas somas faz a versão bottom-up?

49

uma por cada i de 2 a 50

Quantos valores guarda a versão com duas variáveis?

2

em vez de 51

A recursão direta faria mais de 40 mil milhões de chamadas.

3

PARTE 3

O princípio da optimalidade

Soluções ótimas feitas de peças ótimas.

O passo crucial e o princípio que o sustenta

O PASSO CRUCIAL

Derivar uma recorrência que relacione a solução do problema com as soluções dos seus subproblemas mais pequenos.

PRINCÍPIO DA OPTIMALIDADE

Uma solução ótima de um problema de otimização é composta por soluções ótimas das suas sub-instâncias.

Exemplo: o caminho mais curto de Coimbra a Lisboa



Se um troço não fosse o mais curto, trocávamo-lo por outro e a viagem toda ficava mais curta. O princípio verifica-se muito frequentemente (mas há exceções).

4

PARTE 4

O problema do troco

Programação dinâmica em ação.

Pensar na última moeda

Dar troco de n com o menor número de moedas de valores $d_1 < d_2 < \dots < d_m$ (com $d_1 = 1$ e moedas ilimitadas).

A ideia: qual foi a última moeda?

montante 5

+

1

= 6

montante 3

+

3

= 6

montante 2

+

4

= 6

Para $n = 6$ com moedas 1, 3, 4: três hipóteses para a última moeda.

$F(n)$ = número mínimo de moedas para n

$$F(n) = \min \{ F(n - d_j) \} + 1$$

para todas as moedas com $d_j \leq n$

$$F(0) = 0$$

" $+ 1$ " é a última moeda; o mínimo escolhe a melhor forma de fazer o resto.

Preencher a tabela para n = 6, moedas 1, 3, 4

n	moeda 1: F[n-1]	moeda 3: F[n-3]	moeda 4: F[n-4]	F[n] = mín + 1
1	F[0]=0	—	—	1
2	F[1]=1	—	—	2
3	F[2]=2	F[0]=0	—	1
4	F[3]=1	F[1]=1	F[0]=0	1
5	F[4]=1	F[2]=2	F[1]=1	2
6	F[5]=2	F[3]=1	F[2]=2	2

A tabela final

0	1	2	3	4	5	6
0	1	2	1	1	2	2

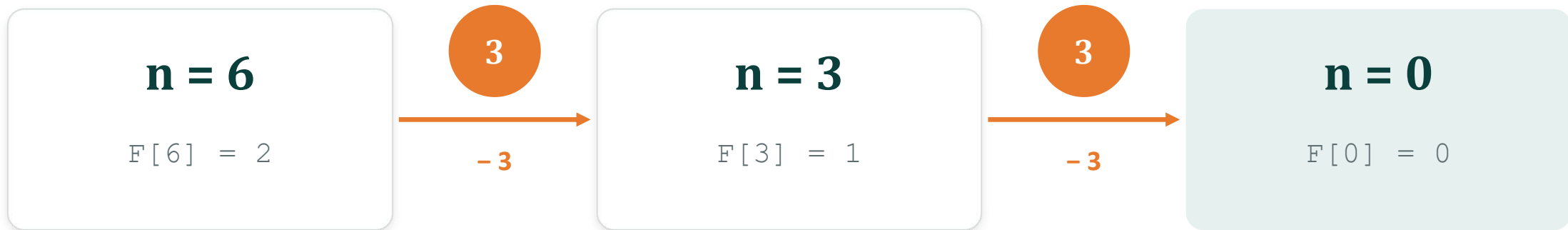
Resposta

F[6] = 2

bastam 2 moedas para 6 cêntimos

Seguir o rasto dos mínimos

Voltamos para trás, vendo em cada passo que moeda produziu o mínimo:



Solução ótima

3 3 = 6 com 2 moedas

Dica de implementação

Guarde numa segunda tabela, para cada n , a moeda que deu o mínimo. No fim, basta seguir essa tabela de n até 0.

ChangeMaking($D[1..m]$, n)

ChangeMaking($D[1..m]$, n)

$F[0] \leftarrow 0$

para $i \leftarrow 1$ até n :

$temp \leftarrow \infty$; $j \leftarrow 1$

 enquanto $j \leq m$ e $i \geq D[j]$:

$temp \leftarrow \min(F[i - D[j]], temp)$

$j \leftarrow j + 1$

$F[i] \leftarrow temp + 1$

devolver $F[n]$

$D[1..m]$ em ordem crescente, com $D[1] = 1$.

Tempo

$O(n \cdot m)$

n montantes $\times m$ moedas

Espaço

$O(n)$

a tabela $F[0..n]$

Agora é consigo



Fibonacci



Implemente em Java a solução de programação dinâmica para a sequência de Fibonacci.

Extra: compare o tempo da recursão direta com o da tabela para $n = 40$.



Problema do troco



Implemente em Java a solução de programação dinâmica para o problema do troco.

Extra: devolva também quais as moedas usadas. Teste: $n = 6$, moedas 1, 3, 4 $\rightarrow$ 3 + 3.



Mini-projeto 1



Implemente em Java a solução de programação dinâmica para o problema do mini-projeto 1.

Comece por escrever a recorrência: é o passo crucial.

Três ideias a reter

1

Resolver uma vez, guardar

Subproblemas sobrepostos resolvem-se uma só vez e ficam numa tabela.

2

Bottom-up ou top-down

Tabela preenchida do mais pequeno para o maior, ou recursão com memória.

3

A recorrência é tudo

Escreva-a primeiro. No troco: $F(n) = \min F(n - d_j) + 1$, em $O(n \cdot m)$.

Força bruta experimenta tudo; programação dinâmica lembra-se do que já fez.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.^a ed.). Pearson. Capítulo 8: Dynamic Programming.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt